

SMALL LANGUAGE MODEL CHALLENGE

Parameter-Matched SwiGLU and Capacity Scaling

Course: DASE7506

Student ID: 3036781272

Name: Li Zhoutong

1. Summary

I first reproduced the supplied causal language-model baseline and then changed one component at a time. My main modification replaces the GELU feed-forward sublayer with a parameter-matched SwiGLU gate. At the same training budget, validation BPB improved from 2.071083 to 2.010852. I then trained a wider version of the selected design for longer. The frozen final model reached 1.710366 validation BPB and 1.738670 test BPB, which is 17.26% lower than the reproduced baseline test result. I use the matched run to analyse the architectural change and the wider run only as the final quality-cost trade-off.

Final test BPB	CPU score time	Peak RAM	Inference assets
1.738670	19.93 s (1.48x)	1.5333 GiB	8.5041 MiB

2. Task and baseline

The task is next-token prediction on the supplied WikiText-2 benchmark, a corpus introduced by Merity et al. [3], using the fixed training-fitted BPE-2048 tokenizer and evaluator supplied by the course [1]. Evaluation uses independent causal windows of 256 targets and reports summed negative log probability in bits divided by the raw UTF-8 byte count. Lower BPB is better. I used validation for development and model selection and evaluated test only after freezing the final predictor.

The baseline is a small decoder-only Transformer in the general architecture of Vaswani et al. [4]. It has four pre-LayerNorm blocks, width 128, four attention heads, learned absolute positions, residual connections, tied token/output embeddings and a 4x GELU MLP. It contains 1,088,256 parameters and is trained from random initialization for 1,200 updates with batch size 32, corresponding to 9,830,400 processed targets. Importantly, the supplied attention is already multi-head and causally masked; changing it to 'multi-head masked attention' would therefore not create a new mechanism.

I reproduced the baseline on Windows 11, Python 3.12.13 and PyTorch 2.7.1+cpu using four threads on an AMD Ryzen 7 4800HS. Training took 769.99 s; validation BPB was 2.071083 and test BPB was 2.101260. This matches the staff reference of approximately 2.10 BPB.

2.1 Baseline components and possible improvements

I divided the baseline into five parts before choosing a modification: embeddings and positions, causal attention, the feed-forward sublayer, normalisation/residual connections, and the training configuration.

This made it easier to distinguish an architectural change from simply using more parameters or more updates.

Embeddings and positions. The learned absolute position table could be replaced by a relative or rotary scheme, but the evaluator fixes the context to 256 tokens, so long-context extrapolation is not the main limitation in this project.

Causal attention. Head count, model width and the attention implementation can be changed. However, the existing model already uses scaled dot-product multi-head attention with a causal mask. A more complicated attention change would require additional controls and was harder to isolate within the time available.

Feed-forward sublayer. This sublayer applies the same nonlinear feature transformation independently at every token and accounts for a substantial part of each block's parameters. Its activation and hidden-width allocation can be changed without touching tokenisation, causal masking or evaluation. This made it the most practical component for a controlled first experiment.

Normalisation, residuals and training. Alternatives such as RMSNorm, different regularisation, learning-rate schedules, model width, depth and training duration may also improve BPB. These choices interact with optimisation, so I kept them fixed in the mechanism experiment and changed capacity and duration only after selecting SwiGLU on validation.

3. Method

3.1 Controlled mechanism: parameter-matched SwiGLU

I chose SwiGLU because it was a small change that I could explain and test with a fair ablation. Shazeer [2] reports that GLU variants can improve Transformer feed-forward sublayers. My expectation was that a learned gate could decide which candidate features to pass for each token, whereas the baseline GELU MLP transforms only one projected path. I did not assume that the published result would automatically transfer to this much smaller model, so the parameter-matched run was necessary.

The baseline MLP has one nonlinear feature path:

$$\text{GELU MLP}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2$$

I replace it with a gated path:

$$\text{SwiGLU}(x) = W_o [\text{SiLU}(W_g x + b_g) * (W_v x + b_v)] + b_o$$

The gate branch can suppress or pass each candidate value feature at every token position. Directly retaining hidden width 512 would add many parameters because SwiGLU has gate, value and output projections. I therefore used hidden width 341. The matched student has 1,088,424 parameters, only 168 (0.015%) more than baseline. Attention, LayerNorm, positions, residual layout, initialisation, optimiser, seed and sampling are held fixed. The original GELU model is therefore the removal ablation for the gate. This setup follows the SwiGLU form described in [2], adapted to the supplied model interface.

3.2 Final quality/cost candidate

After the controlled SwiGLU model improved validation, I froze the mechanism choice and allocated additional allowed training/model capacity to the final candidate: width 192, six heads, four blocks and SwiGLU hidden width 512. It has 2,223,232 parameters and trains for 3,600 updates (29,491,200 targets). This model intentionally changes both width and training duration, so it is a final quality/cost optimization rather than evidence that isolates SwiGLU.

4. Experimental design

Run	Width / heads	Steps	Targets	Parameters	Purpose
GELU baseline	128 / 4	1,200	9,830,400	1,088,256	Ablation / reference
Matched SwiGLU	128 / 4	1,200	9,830,400	1,088,424	Mechanism test
Final SwiGLU	192 / 6	3,600	29,491,200	2,223,232	Frozen score

All runs use seed 17, context 256, AdamW with the supplied schedule, gradient clipping and only the supplied training text. The fixed evaluator and tokenizer are unchanged. The 1,200-step pair is the required same-target comparison and key-mechanism ablation. The final model is selected solely because its 1.710366 validation BPB is lower than both controlled runs.

5. Results

Method	Val BPB	Test BPB	Train time	CPU test	Checkpoint fingerprint (short SHA-256)
GELU baseline	2.071083	2.101260	769.99 s	13.46 s	7d869d...6d3f
Matched SwiGLU	2.010852	Not run	724.22 s	Not run	a555a3...3c94
Final SwiGLU	1.710366	1.738670	3,483.08 s	19.93 s	373b88...7dcc

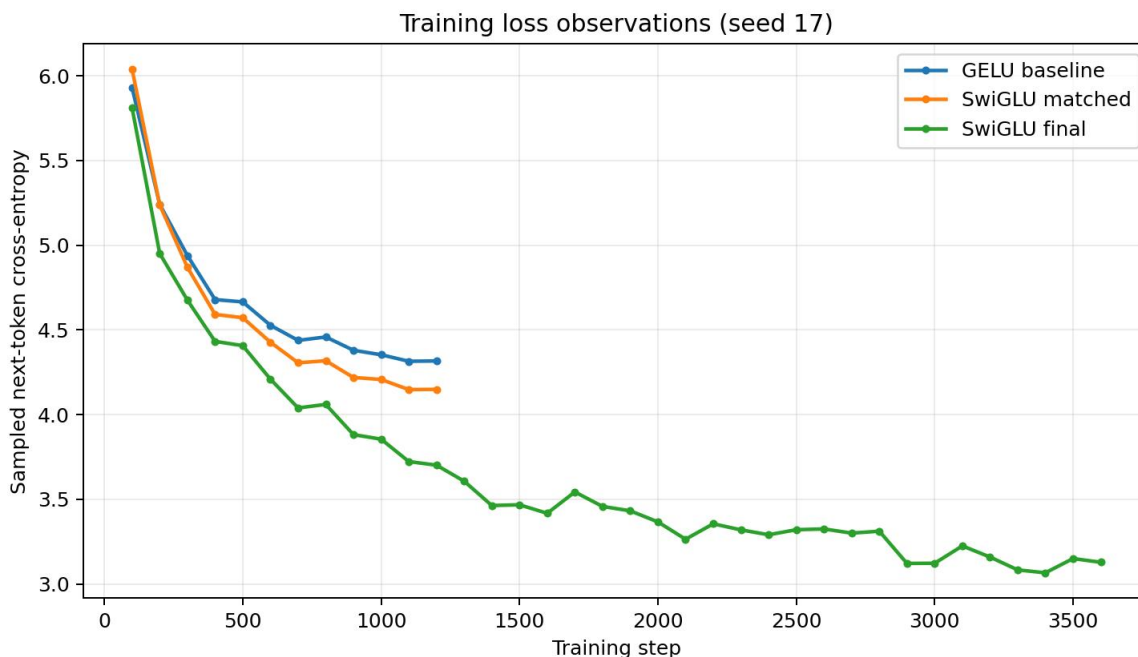


Figure 1. Logged training losses. The matched pair shares the same step budget; the wider final run continues to 3,600 updates.

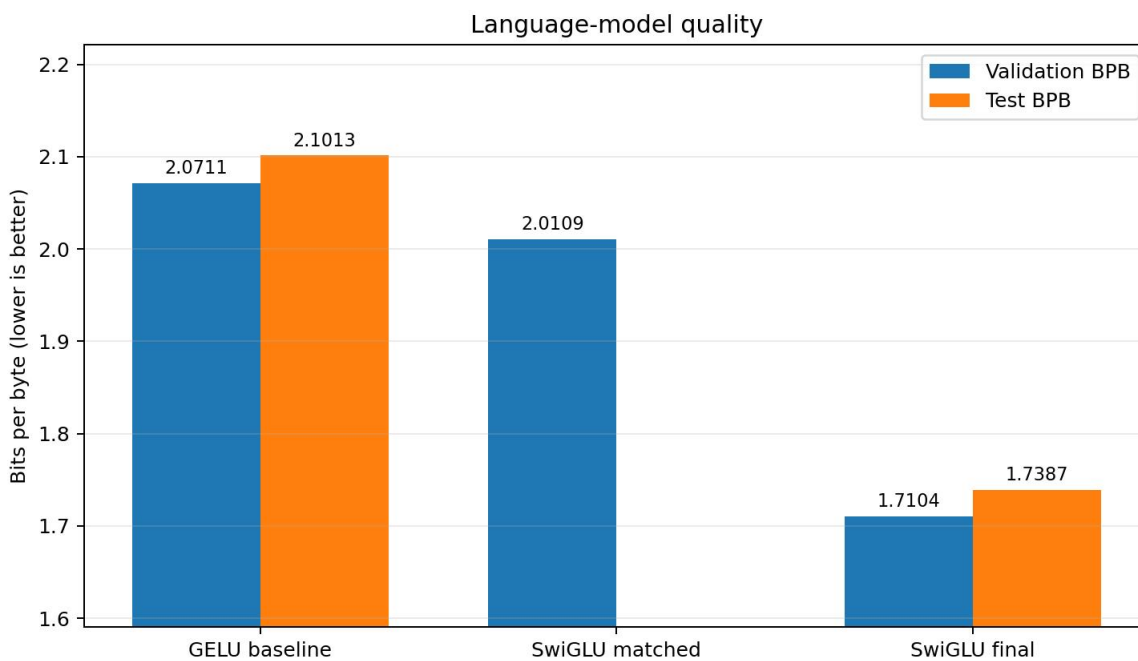


Figure 2. Validation and frozen test BPB. Lower is better; unselected models were not evaluated on test.

The matched experiment is the clearest evidence for the proposed mechanism. Both models see exactly 9,830,400 targets, and their parameter counts differ by only 0.015%. The 0.060232 validation-BPB reduction therefore cannot be explained by a larger training set or a meaningfully larger model. The matched model was not evaluated on test because it was a validation-only ablation, not the frozen submission candidate.

The final model improves validation by 0.360717 BPB (17.42%) and test by 0.362590 BPB (17.26%) relative to baseline. It uses 2.04x parameters, 3x processed targets and 4.52x training time. The inference increase is smaller: 19.93 s is 1.48x the locally reproduced baseline CPU scoring time. A repeated resource-instrumented pass reproduced exactly the same test BPB in 20.06 s, with 1.5333 GiB peak process-tree working set. Required checkpoint, code and configuration assets total 8.5041 MiB. The model therefore stays below all three limits, although the quality gain required noticeably more training.

6. Analysis and trade-offs

6.1 What the controlled comparison establishes

At the same 9,830,400 targets, validation BPB decreases by 0.060232 (2.91% relative) when GELU is replaced by parameter-matched SwiGLU. In the logged training curves, the two runs are close early in training; the SwiGLU loss becomes consistently lower after roughly 300 updates and stays lower through step 1,200. Because the final validation score agrees with the direction of the training curves, I interpret this as evidence that gating used the same feed-forward parameter budget more effectively in this run.

What this comparison does not establish is equally important: it uses only one seed and one small model, and it does not compare SwiGLU with every other possible activation or training schedule. The result supports this design choice for the submitted setting rather than a universal claim that SwiGLU always improves language models.

The matched run took 724.22 s versus 769.99 s for baseline, but this single local timing should not be interpreted as an inherent SwiGLU speed advantage; background load and kernel variation were not controlled across repetitions.

6.2 Final score versus cost

The final model improves validation by 0.360717 BPB (17.42%) and test by 0.362590 BPB (17.26%) relative to baseline. It uses 2.04x parameters, 3x processed targets and 4.52x training time. The inference trade-off is smaller: 19.93 s is 1.48x baseline CPU scoring time. A repeated resource-instrumented pass reproduced the exact test BPB in 20.06 s, with 1.5333 GiB peak process-tree working set. Required checkpoint/code/config assets total 8.5041 MiB. These are below the 5x time, 4 GiB RAM and 64 MiB asset limits.

Constraint	Limit	Measured	Status
CPU scoring	<= 5x baseline	1.48x	Pass
Peak evaluation RAM	<= 4 GiB	1.5333 GiB	Pass
Inference assets	<= 64 MiB	8.5041 MiB	Pass

6.3 Limitations

The main limitation is that I ran one seed for each complete experiment, so I cannot estimate run-to-run variance. The matched evidence is also limited to one small scale and one corpus. For the final model,

width and training duration change together, so their individual contributions are unknown. Given more time, I would repeat the matched pair with additional seeds and add separate width-only and training-duration-only controls. I also did not run the matched candidate on test, because development and selection were restricted to validation by design.

7. Reproducibility

From the code directory, install the pinned dependencies and run the contract tests before training:

```
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
```

Controlled SwiGLU training:

```
python train.py --implementation student --config configs/swiglu_128.json --device cpu --
precision fp32 --threads 4 --seed 17 --steps 1200 --batch-size 32 --run-dir runs/swiglu128-
fair-s17
```

Final training and frozen evaluation:

```
python train.py --implementation student --config configs/swiglu_192.json --device cpu --
precision fp32 --threads 4 --seed 17 --steps 3600 --batch-size 32 --run-dir runs/swiglu192-
final3600-s17
python evaluate.py --checkpoint runs/swiglu192-final3600-s17/checkpoint.pt --device cpu --
precision fp32 --threads 4 --split test
```

SHA-256 is a file fingerprint, not an accuracy score. If even one byte of a checkpoint changes, its SHA-256 value changes. Recording the fingerprint lets the evaluator check that the downloaded file is exactly the checkpoint used for the reported result. The results table shortens each value for readability: 7d869d...6d3f identifies the baseline checkpoint, a555a3...3c94 identifies the matched validation-only checkpoint, and 373b88...7dcc identifies the frozen final checkpoint. Only the final checkpoint is needed for the ranked submission.

The complete frozen-final checkpoint SHA-256 is 373b884e4daec5c99ad2e0645106ead62580b68c2b07a0eb2220e3c7c1c87dcc. The supporting student_model.py SHA-256 is b8bd352f7cb14832536882a0b4fea4308e3d8371443cadef6a5179ba3af63ad4.

8. Conclusion

The controlled experiment answers my main question: at nearly the same parameter count and exactly the same target budget, the SwiGLU version achieved lower validation BPB than the supplied GELU MLP. The wider, longer-trained version then reached 1.738670 test BPB while remaining within the evaluation limits. The experiments do not show that gating alone caused the entire final improvement, but they separate the evidence for the gate from the later decision to spend more model and training capacity.

References

[1] DASE7506. MP1 Small Language Model Challenge starter package and evaluation protocol, 2026.

- [2] N. Shazeer. GLU Variants Improve Transformer. arXiv:2002.05202, 2020.
<https://arxiv.org/abs/2002.05202>
- [3] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer Sentinel Mixture Models. arXiv:1609.07843, 2016. <https://arxiv.org/abs/1609.07843>
- [4] A. Vaswani et al. Attention Is All You Need. arXiv:1706.03762, 2017.
<https://arxiv.org/abs/1706.03762>